

Tarea 1: Simulando EloTelTag y Aplicación Find My

Lea detenidamente la tarea. **Si algo no lo entiende, consulte. Si es preciso, se incorporarán aclaraciones al final.** Esta interacción se asemeja a la comunicación entre desarrolladores de software y un cliente con el fin de aclarar requerimientos de una aplicación.

1. Objetivos de la tarea

- Modelar objetos reales como objetos de software.
- Ejercitar la creación y extensión de clases dadas para satisfacer nuevos requerimientos.
- Reconocer clases y relaciones entre ellas en código fuente Java.
- Ejercitar la compilación y ejecución de programas en lenguaje Java desde una consola de comandos.
- Ejercitar la configuración de un ambiente de trabajo para desarrollar aplicaciones en lenguaje Java. Se puede trabajar con un editor tipo "Sublime" o con un IDE (Integrated Development Environment). IntelliJ es el IDE sugerido tanto para esta tarea como la Tarea 2.
- Ejercitar la entrada y salida de datos en Java.
- Manejar proyectos vía Git (**voluntario para esta tarea**).
- Conocer el formato .csv y su importación desde una planilla electrónica.
- Ejercitar la preparación y entrega de resultados de software (creación de makefile, readme, documentación).
- Familiarización con desarrollos "iterativos" e "incrementales" (o crecientes).

2. Descripción General

Esta tarea está **inspirada** en el producto [AirTag](#). Un AirTag es un dispositivo localizador de objetos personales (llaves, mochilas, maletas, etc.). Lo señalado en esta tarea tiene precedencia respecto de lo que usted sepa o lea sobre AirTag; sin embargo, ante una especificación incompleta, la idea es aclararla basada en la operación de estos dispositivos. Un EloTelTag es un dispositivo AirTag con funcionalidades reducidas y se reporta vía cualquier celular.

Un EloTelTag se puede guardar en una mochila, maleta, cartera, llavero, etc. Cada persona poseedora (dueño/a) de un EloTelTag además debe poseer al menos un celular y también podría tener un tablet. Cada EloTelTag, tablet y celular tiene un nombre (por ejemplo para indicar qué identifica -celular, Tablet, maleta, llaves, etc.) y un identificador de su dueño/a. Para efectos de esta tarea sólo los celulares pueden reportar su posición (tienen un [GPS](#), Global Positioning System). Los EloTelTag y tablets se comunican con cualquier celular cercano, esto es si su distancia es inferior a 10 [m], para reportar su localización (en realidad la del celular), su nombre y el nombre de su dueño/a. Así, mientras los celulares pueden reportar a la ETnube su posición, los EloTelTag y los tablets lo hacen a través de un celular cercano perteneciente a la misma u otra persona. Ver Figura 1.

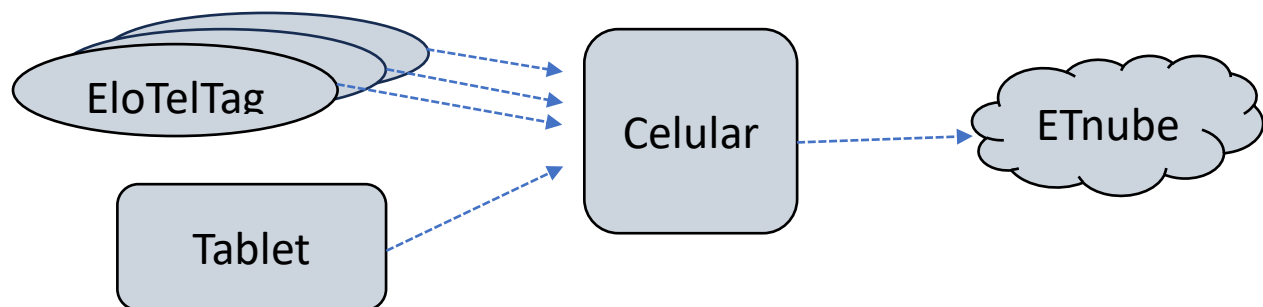


Figura 1: EloTelTag rastreadores de ítems, dispositivos Tablet y Celular, y ETnube registradora de localizaciones

Es este mundo simplificado, FindMy es una aplicación existente en celulares y tablets que muestra la posición (x, y) de los ítems de una persona (aquellos con EloTelTag) y de los dispositivos de la persona (celular y tablet). Par ello, la funcionalidad FindMy accede a la nube ETnube y obtiene la posición de todos artículos asociados a una misma persona. Ver Figura 2.

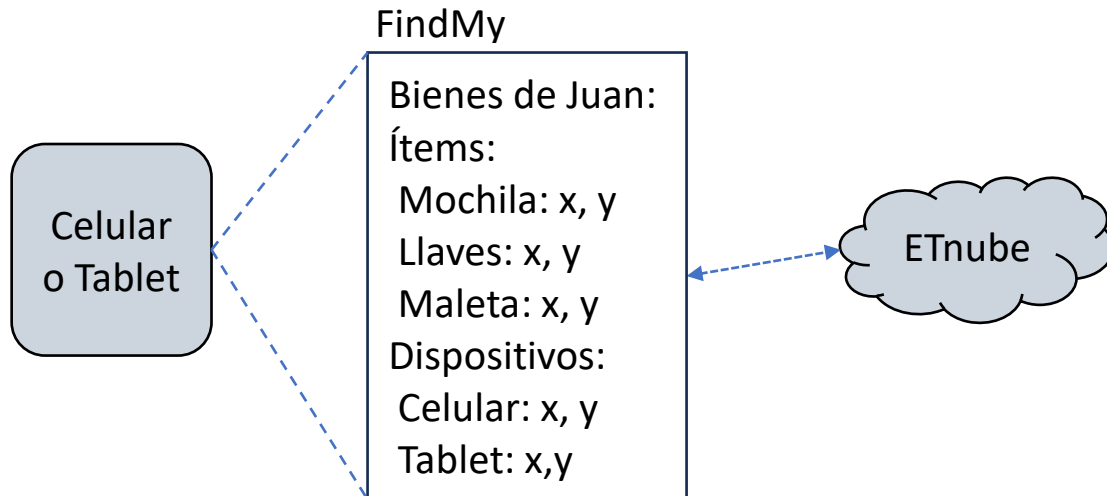


Figura 2: Aplicación FindMy de celulares y tables mostrando localizaciones de una persona

Hasta aquí se ha descrito cómo funciona una tecnología de localización simple inspirada en la tecnología AirTag comercial.

Ahora procederemos a describir el modelo de software de esta tarea para los objetos del mundo real que dan vida a este servicio de localización.

2.1 Modelo básico de EloTelTag, Celular, Tablet y ETnube

Clases de objetos de esta solución son, entre otros: EloTelTag, Celular, Tablet, y ETnube.

- *EloTelTag*: debe tener, al menos, un nombre del tag y el nombre de su dueño/a. El nombre del dueño/a más el nombre del tag debe ser único entre todos los equipos (EloTelTag, celular o tablet).
- *Celular*: debe tener, al menos, el nombre de su dueño/a y podrá reportar su posición a la ETnube. La clase Celular debe también ofrecer el servicio FindMy que muestra la localización de todos los equipos de la persona. Para ello Celular debe tener una referencia al objeto ETnube.
- *Tablet*: tiene atributos similares que un EloTelTag, pero además ofrece el servicio FindMy de manera similar a la clase Celular.
- *ETnube*: Registra la última posición informada, por cualquier celular, de cualquier EloTelTag y cualquier Tablet. ETnube también puede ser consultada por las posiciones de todos los equipos pertenecientes a una persona.

Notar que estos modelos dejan fuera todas las funciones típicas de celulares y tablets pues esta tarea sólo se focaliza en el servicio de localización de equipos.

Notar que tags y tablets no saben de su posición, al igual que una persona no sabe de su peso a menos que use una pesa; sin embargo, incluiremos en éstos sus coordenadas, como es común incluir el peso como atributo de una persona. A diferencia de los celulares, tags y tablets indirectamente reportan a ETnube la localización de cualquier celular cercano y no sus coordenadas que usaremos para establecer su posición y movimiento en un territorio plano. Como parte del modelo, se considerará que tags, tablets y celulares también se pueden desplazar. En la realidad

algo los mueve -persona en su mochila o llavero, maletas en avión, auto, etc.-, para no incluir mayor complejidad en el modelo, se considerará que estos equipos se desplazan según alguna ruta definida por un archivo de entrada.

2.2 Funcionamiento del Programa

La simulación es configurada vía un archivo de entrada (por ejemplo config.txt) pasado como argumento del programa. Este archivo especifica los/as propietarios/as, sus equipos y sus coordenadas iniciales. El formato para las líneas de este archivo es como sigue:

```
<número de personas>  
<nombre 1º persona> <número de EloTelTags> (0 | 1)  
<celular_x> <celular_y>  
(<nombre EloTelTag> <tag_x> <tag_y> ) *  
[<tablet_x> <tablet_y>]
```

Las líneas previas, excepto la primera, se repiten según el número de personas consideradas. (0 | 1) en la primera línea de definición de una persona indica si ésta tiene o no un Tablet.

<tag_x> e <tag_y> representan las coordenadas iniciales del tag.

*: significa repetir 0 o más veces. [a] significa que a es opcional.

Un ejemplo para este archivo es:

```
3  
Pedro 2 1  
3.4 10.6  
maleta 5.0 7.8 mochila 12.6 1.2  
3.5 9.4  
Juan 1 0  
50.1 63  
llaves 25.1 25.8  
Diego 2 1  
35 35  
llaves 0.1 0.1 mochila 40.5 43.1  
-21 31.0
```

Una vez creados los objetos a partir del archivo de configuración, el programa acepta los siguientes comandos ingresados a través de otro archivo como segundo parámetro del programa (ejemplo move.txt). El formato para este archivo es:

```
(<nombre del objeto> (<  $\Delta x$  > <  $\Delta y$  > | FindMy ) <newline>)+
```

+ significa una o más líneas. (a | b) significa a o b, pero no ambos.

Un ejemplo de este archivo es (sólo las primeras líneas):

```
Juan.celular 0.5 -0.8  
Juan.llaves 1 2.0  
Diego.tablet -0.6 -0.9  
Diego.tablet FindMy  
Pedro.celular 10 10  
Juan.celular FindMy
```

Con el comando FindMy, el programa muestra una salida por pantalla similar a la indicada en Figura 2, con la última posición **reportada** por algún celular para cada equipo de la persona propietaria. Adicionalmente, el programa genera el archivo "output.csv". Luego de la configuración de los objetos y después de cada comando, el programa escribe en este archivo la última posición x, y reportada para cada equipo configurado. La primera línea describe cada una de las columnas. A modo de ejemplo:

Step	Pedro.celular.x .y			Pedro.maleta.x .y			Pedro.mochila.x .y		Pedro.tablet.x .y
0	3.4	10.6	5.0	7.8	12.6	1.0	3.5	9.4	
1	3.4	10.6	5.0	7.8	12.6	1.0	3.5	9.4	
2	3.4	10.6	5.0	7.8	12.6	1.0	3.5	9.4	
3	3.4	10.6	5.0	7.8	12.6	1.0	3.5	9.4	
4	3.4	10.6	5.0	7.8	12.6	1.0	3.5	9.4	
5									

Nota: sólo se muestra los equipos de Pedro, cada línea debería tener todos los equipos. Usar <TAB> como separador.

2.3 Ejecución del Programa

El programa será revisado en Aragorn y su ejecución es el tipo:

```
$ java SimulatorTest config.txt move.txt
```

Terminada la ejecución de su programa, importe el archivo output.csv desde una planilla Excel. Usando líneas de distinto color, para cada persona grafique las trayectorias (x,y) de cada equipo. Naturalmente, usted debe hacer esto manipulando un programa tipo Excel, no es algo hecho por su programa.

3 Desarrollo en Etapas

Para llegar al resultado final de esta tarea usted avanzará en pasos "[Iterativos y crecientes \(o incrementales\)](#)" comunes en metodologías de desarrollo de software actuales. Usted y su equipo irán desarrollando etapas donde los requerimientos del sistema final son abordados gradualmente. En cada etapa usted y su equipo obtendrá una solución que funciona para un subconjunto de requerimientos finales o bien es un avance hacia ellos. En AULA se dispondrá el recurso para subir la solución correspondiente a cada etapa del desarrollo. **Su equipo deberá entregar una solución para cada una de las etapas** aun cuando la última integre las primeras. **Los archivos readme.txt, de documentación y eventualmente otros (gráficos) deben ser preparados sólo para la última etapa. Prepare y entregue un makefile para cada una de las etapas.** Esto tiene por finalidad, educar en la metodología de desarrollo iterativo e incremental.

3.1 Primera Etapa: Lectura de archivos de entrada clase ELOTelTags

En esta etapa se deben crear, al menos, las clases T1Stage1 y ELOTelTag. La lectura de los archivos de entrada (por ejemplo config.txt y move.txt) puede mantener el formato pero sólo se considerarán personas propietarias de tags. En esta etapa, no se pide manejar archivos mal configurados.

Para verificar su programa, genere un archivo de salida, pero mostrando la posición de cada tag al inicio y después de cada comando y sin pasar por ETnube. Verifique que la salida sigue la ruta consistente.

A modo de ayuda, [aquí](#) encontrará un código base para esta etapa. Usted no está obligado/a a usarlo, es una recomendación, otros diseños son posibles.

Verifique que su programa corre en Aragorn.

Nota: puede ser de utilidad este [diagrama de clases](#).

3.2 Segunda Etapa: Celulares reportan posición a ETnube

Se pide crear -al menos- las clases Celular y ETnube. En esta etapa los archivos de entrada incluyen el celular de cada usuario, su movilidad y los reportes que éstos hacen hacia ETnube cuando existe un tag a menos de 10 [m].

Para verificar su programa hasta aquí, genere el archivo de salida mostrando la posición reportada por celulares para cada tag. Programe esta salida como un servicio de la clase ETnube.

A modo de ayuda, [aquí](#) encontrará un código base para esta etapa. [Diagrama de clases](#).

3.3 Tercera Etapa: Visualizador

En esta etapa la clase Celular podrá recibir comandos de visualización (FindMy). Dado que tanto celulares como tablets ofrecen este servicio, se sugiere implementar la clase Viewer. Cada celular y tablet contará con una instancia de Viewer. Ésta accede a una instancia de ETnube y muestra por pantalla la información de los equipos de la persona propietaria del celular (o Tablet más adelante).

3.4 Cuarta Etapa: Etapa final

Incorpore la clase Tablet y las funcionalidades necesarias para cumplir con la descripción de la tarea dada en las secciones previas.

En esta etapa llame SimuladorTest a su programa ejecutable.

3.5 Extra-crédito: Esta parte es voluntaria, su desarrollo otorga 5 puntos adicionales. Si desarrolla esta parte, indíquelo en su documentación. Agregue el comando “suene”:

<nombre del celular> Sound (<nombre de tag> | <nombre de tablet> <newline>)

Cuando un celular está a menos de 10 [m] de un tag o tablet, con el comando suene, hace que el tag o tablet envíe a pantalla el mensaje “sonando” y el celular muestre en pantalla la distancia y dirección en que se encuentra el equipo buscado. La dirección se reporta como el ángulo medido desde eje x de la ubicación del objeto buscado respecto del celular.

Adicionalmente, se otorgarán 3 puntos adicionales a aquellos equipos que entreguen su tarea utilizando la plataforma [Github](#).

Si bien estos adicionales otorgan puntaje adicional, la nota final se satura al llegar a 100.

4 Entrega

Entregue todo lo indicado en “[Normas de Entrega de Tareas](#)”.

Prepare un [archivo makefile](#) para compilar y ejecutar su tarea en aragorn con rótulo “run”. Además, incluya rótulos “clean” para borrar todos los .class generados. Los comandos usados en cada caso son:

```
$ make /* para compilar */
```

```
$ make run /* para ejecutar la tarea supone que las entradas son config.txt y mov.txt */
```

```
$ make clean /* para borrar archivos .class */
```

En su archivo de documentación (pdf o html) incorpore el diagrama de clases de la aplicación (etapa 4) e imágenes de los gráficos de trayectoria de cada equipo para un usuario que tenga al menos 2 tag, un tablet y un celular.

Ayuda:

1. Se incorporarán aquí según surjan preguntas.